

FSM Interpretation for min-mayus-letter-FPGA

May 11, 2026

Project Context

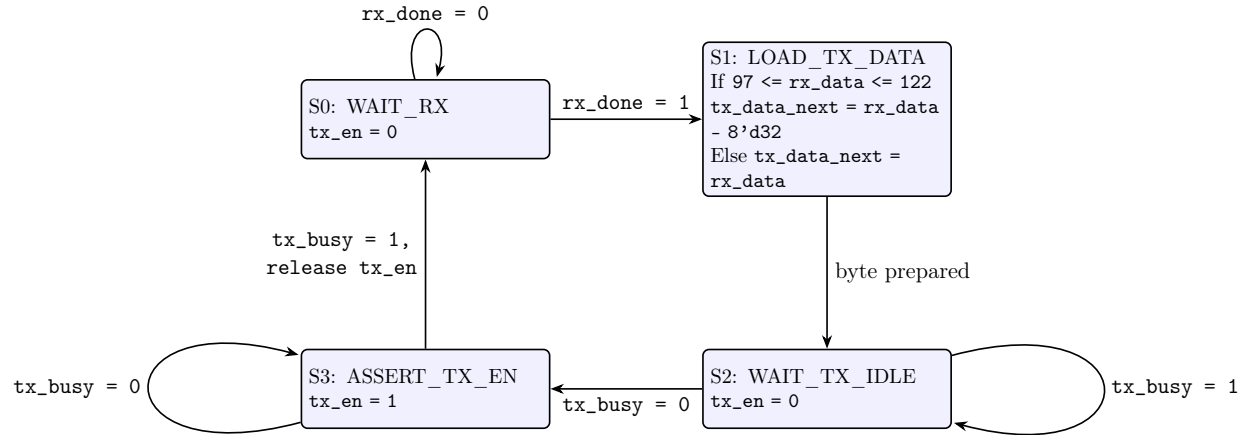
The current `VERIFICATION/min-mayus-letter-FPGA/src/top.v` file only instantiates `uart_rx` and `uart_tx`. The whiteboard process is therefore interpreted as a small supervisory finite-state machine placed above those two UART blocks.

Whiteboard signal	Project signal	Meaning
<code>Rx.done</code>	<code>rx_done</code>	One-cycle pulse when a byte has been received
<code>Rx.rx_data</code>	<code>rx_data</code>	Received byte from <code>uart_rx</code>
<code>Tx.tx_data</code>	<code>tx_data</code>	Byte presented to <code>uart_tx</code>
<code>Tx.tx_start</code>	<code>tx_en</code>	Request to start transmission
<code>Tx.busy</code>	<code>tx_busy</code>	Transmitter is currently sending a frame

Behavior Extracted from the Image

1. Wait until `rx_done = 1`.
2. If `rx_data` is an ASCII lowercase letter ($97 \leq \text{rx_data} \leq 122$), convert it to uppercase by subtracting 32.
3. Otherwise, keep the byte unchanged.
4. Wait until the transmitter is idle (`tx_busy = 0`).
5. Assert `tx_en`.
6. Keep `tx_en` asserted until `tx_busy` goes high, which acknowledges that the UART transmitter accepted the byte.
7. Deassert `tx_en` and return to the receive-wait state.

Proposed Supervisory FSM



State Notes

- **S0: WAIT_RX** corresponds to step 1 in the whiteboard text.
- **S1: LOAD_TX_DATA** implements step 1.1 and decides whether the received byte must be converted from lowercase to uppercase.
- **S2: WAIT_TX_IDLE** matches step 1.2 and prevents a new send request while the transmitter is still busy.
- **S3: ASSERT_TX_EN** matches step 1.3. The state holds the start request until the TX block acknowledges it by raising tx_busy.

Implementation Remark

This FSM is not yet present in the current project top-level. The existing `top.v` exposes the UART interfaces, but a controller register for `tx_data` plus an FSM for `tx_en` would still need to be added if the whiteboard behavior is to become synthesizable RTL.